IN PARTNERSHIP WITH PLYMOUTH UNIVERSITY

Name: Jothikalananthan Janusha
Student Reference Number: 10952971

Module Code**: PUSL2023**	Module Name: **Mobile App Development**
Coursework Title: **PUSL2023 Coursework Final Report Submission**	
Deadline Date: 04/04/2025	Member of staff responsible for coursework: Mr. Diluka Wijesinghe
Programme: BSc (Hons) Software Engineering	

Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook.

Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts.

We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.

Signed on behalf of the group: J.Janusha

Individual assignment: ***I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.***

Signed :

Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence.

I *have used/not used translation software.

If used, please state name of software……

Overall mark _____% Assessors Initials _____ Date_____

*Please delete as appropriateSci/ps/d:/students/cwkfrontcover/2013/14

UNIVERSITY OF PLYMOUTH

PUSL2023
Mobile App Development

Final Report

Pixel Ninja Flutter Game

Group No: 64

Student ID (Plymouth)	Name (as appeared on DLE)	Role	Degree Program
10952715	Don Hettiarachchi	Coding	SE
10952746	Liyana jayasinghe	Coding	SE
10952756	Karunanayaka Jayawardhana	Coding	SE
10952760	Hewawasam Bathika	Coding	SE
10952778	Withana Gayuka	Level design	SE
10952971	Jothikalananthan Janusha	Level design	SE
10953085	Thennakoon Thennakoon	Level design	SE
10955156	Aththanayake Asini Nisansa	Level design	SE

Table of Contents

Chapter 01

1.1 Introduction

Mobile games have become a central aspect of online entertainment, and the demand for interactive, high-performance experiences has grown intensely. The Pixel Ninja Flutter Game was developed to address shortcomings common in most mobile platformers today, such as performance inefficiencies, unbalanced gameplay mechanics, and low incorporation of present cloud-based services. The game was created using Flutter, which enabled cross-platform development, and the Flame game engine, which provided an efficient framework for rendering and physics-based interaction. Firebase services were also integrated to enable secure user authentication, real-time scores, dynamic leaderboards, and persistent cloud storage. The entire game design, ranging from the very first sequence in the main module to the intricate state management of the player character, was created to present a seamless and exciting experience during the gameplay.

1.2 Existing Systems and Problem Definition

1.2.1 Existing Systems

A recent survey of mobile platformers showed while many games—which are often created with tools like Unity or Unreal, looked great, there were several pitfalls common among them

- **Performance Inefficiencies-** Many games demanded high computational resources, leading to weak performance on low-end devices.
- **Gameplay Imbalance-** Some games were accused of over-complicating controls or having too simplistic gameplay to engage consumers.
- **Limited Cloud Integration-** Recent games rarely had robust executions of real-time score updates, dynamic leaderboards, or secure management of user data.
- **Rigid Level Designs-** All platformers were found to follow fixed level design, without dynamism enough to sustain long-term interest among players.

These challenges were followed by popular games such as Super Mario Run and Geometry Dash, which, despite their commercial success, faced criticism for inconsistent performance and inflexible game mechanics.

1.2.2 Problem Definition

The Pixel Ninja Frog Flutter Game was conceptualized to bridge the identified gap by addressing the following issues,

- **Optimization and Accessibility-** The game was designed to be lightweight and efficient, ensuring smooth performance across a wide range of mobile devices.
- **Balanced and Engaging Gameplay-** A central focus was placed on developing intuitive controls and realistic physics, including proper implementation of gravity, jump force, and collision responses
- **Advanced Technology Integration-** The project utilized Flutter for UI and cross-platform compatibility, the Flame engine for game logics, and Firebase for cloud-based services such as authentication, score management, and persistent data storage.
- **Dynamic Interactivity-** The game incorporated responsive animations, state-based character management, and robust collision detection to create a dynamic and immersive experience that adapted to player inputs in real time.

1.3 Project Aims and Objectives

1.3.1 Aim

The primary aim of the project was to develop a highly featured, optimized, and engaging 2D platformer mobile game with the latest development frameworks and tools. It was a question of leveraging Flutter as a cross-platform development tool, Flame game engine as a rendering engine and game engine, and Firebase to achieve integration with the cloud securely.

1.3.2 Objectives

The objectives of the project were defined as follows,

Implementation of Core Mechanics-

- A detailed character control system was developed, including movement, jumping, and state transitions (idle, running, jumping, falling, hit, appearing, disappearing).
- Sprite animations were dynamically loaded from sprite sheets, ensuring smooth visual transitions during gameplay.

Physics and Collision Management-

- Realistic physics were implemented using constants for gravity, jump force, and terminal velocity.
- Robust collision detection routines were established to manage interactions between the player and game objects (e.g., fruits, saws, checkpoints, chickens, spikes).

Firebase Integration-

- Firebase was employed to enable secure user authentication (supporting both Google and email/password sign-in).
- Real-time score tracking and dynamic leaderboard updates were implemented using Firebase's real-time database.
- Cloud storage was configured to persist user progress and game data across sessions.

User Interface and Experience-

- A polished UI was designed using Flutter's Material widgets, featuring a splash screen, main menu, game over screen, settings, and profile management.
- Visual elements such as a dynamic loading screen, themed backgrounds, and animated UI components contributed to an immersive user experience.

Level Design and Progression-

- Multiple levels with escalating difficulty were developed using tiled maps and custom collision blocks to ensure an engaging progression.

Audio-Visual Enhancements-

- Sound effects (e.g., jump, hit, collection sounds) were incorporated to provide immediate feedback and enhance the overall gaming experience.

1.4 Scope of the Project

1.4.1 Included Features

The scope of the project encompassed the following features:

Game Engine and Framework-

- The project utilized Flutter for UI and Flame for game mechanics, capitalizing on their ability to deliver efficient cross-platform performance.

Core Gameplay Mechanics-

- Detailed character control with state-based animations and realistic physics.
- Interaction with various game objects, including obstacles, collectibles, and enemies, managed through collision detection and response routines.

Level Design-

- Multiple levels were created using Tiled maps, with custom-defined spawn points for players and objects.
- Dynamic level progression was achieved through the implementation of a level manager that loaded and transitioned between levels.

Firebase Services-

- User authentication, real-time score tracking, and dynamic leaderboard functionalities were implemented to enhance player engagement.

User Interface-

- A visually appealing main menu, in-game HUD (including joystick controls and jump buttons), and settings interface were developed to ensure intuitive navigation.

Audio and Visual Feedback:

- Sound effects and background music were integrated to enrich the gaming experience, synchronized with user actions and game events.

Chapter 02

2.1 Requirement Gathering Techniques

A comprehensive set of requirement gathering techniques was employed to capture both technical and user-centric specifications for the game. The following methods were used,

- **Surveys and Questionnaires-**

  Feedback was collected from potential users to gauge expectations regarding gameplay mechanics, visual design, and overall user experience.

- **Interviews**

  Direct interviews with mobile gamers were conducted to identify common issues and desired features in platformer games.

- **Competitive Analysis-**

  Existing popular platformers, such as Super Mario Run and Geometry Dash, were analyzed to determine areas of improvement, especially concerning performance optimization and user engagement.

- **Brainstorming Sessions-**

  Regular team discussions facilitated the refinement of game mechanics, level design, and UI elements. These sessions significantly influenced the modular design of the codebase.

- **Prototyping and Iterative Testing-**

  Early prototypes were developed using Flutter's hot reload feature, enabling rapid testing and iterative improvements of gameplay elements.

- **User Story Mapping-**

  Player personas were created and mapped against different game interactions to ensure that the final product would address a wide range of user needs.

2.2 Functional and Non-Functional Requirements

2.2.1 Functional Requirements

The game was designed to satisfy the following functional requirements:

User Authentication-

- Secure sign-in options were implemented using Firebase Authentication, supporting both Google and email/password methods.

Character Movement and Animation-

- The player character could move, jump, and perform state transitions (idle, running, jumping, falling, hit, appearing, disappearing) based on input events, as managed by the Player class.

Collision Detection and Response-

- Robust collision handling routines were developed to manage interactions with various objects, including fruits, saws, checkpoints, chickens, and spikes.
- Specific behaviors were triggered upon collision (e.g., respawning, score updates, triggering animations).

Level Progression-

- Multiple levels were created, each with unique spawn points, obstacles, and environmental designs.
- A level manager was implemented to transition between levels based on player progress.

Score Management and Leaderboard-

- Real-time score updates and leaderboard functionalities were integrated using Firebase's real-time database.

User Interface (UI)-

- A series of UI components were developed, including a splash screen, main menu, in-game HUD (with joystick and jump button components), profile screens, and a game completion screen.

Power-Ups and Tutorial Mode-

- Additional gameplay features, such as power-ups and tutorial modes, were provided to enhance user engagement and ease of entry for new players.

2.2.2 Non-Functional Requirements

To ensure that the game was not only functional but also robust and user-friendly, the following non-functional requirements were addressed,

Performance Optimization-

- The game was optimized for smooth operation on a range of mobile devices by implementing efficient asset loading, rendering techniques, and physics calculations.

Cross-Platform Compatibility-

- A consistent user experience was maintained across Android and iOS platforms by leveraging Flutter's cross-platform capabilities.

Security-

- Secure storage and transmission of user data, particularly in relation to authentication and score management, were prioritized.

User Experience (UX)-

- The UI was designed to be intuitive and visually cohesive, minimizing the learning curve and ensuring engaging interactions.

Scalability-

- The game architecture was designed to be extensible, allowing for future updates such as additional levels, new character skins, and expanded gameplay features.

Accessibility-

- Options for adjustable controls, subtitles, and customizable audio/visual settings were provided to accommodate a diverse user base.

2.3 Features of the Application

2.3.1 Core Gameplay Features

Character Controls and Animation-

- The Player class managed detailed character animations, transitioning between states such as idle, running, jumping, falling, and hit.
- Animations were dynamically loaded from sprite sheets, ensuring a fluid visual experience that accurately reflected player input.

Physics-Based Movement-

- Realistic physics were applied, including the simulation of gravity, jump forces, and terminal velocity.
- Collision detection routines ensured that interactions with game objects resulted in appropriate responses (e.g., respawning or triggering checkpoints).

Dynamic Collision Handling-

- Interactions with game objects (such as fruits, saws, spikes, checkpoints, and enemy characters like chickens) were managed through specialized collision components, with corresponding actions (score updates, animations, or state changes) triggered on impact.

Level Design and Environmental Dynamics-

- Levels were created using tiled maps and included dedicated layers for backgrounds, collisions, and spawn points.
- Dynamic elements such as scrolling backgrounds and moving obstacles (saws, enemy chickens) were implemented to provide varied challenges.

Audio-Visual Feedback-

- Sound effects were integrated to provide immediate feedback on actions (e.g., jump sounds, collision sounds, and collectible effects), and dynamic UI animations enhanced the visual appeal.

2.3.2 Firebase Integration

User Authentication-

- Secure login functionality was provided through Firebase, supporting both Google sign-in and email/password methods.

Real-Time Score Management-

- The game utilized Firebase's real-time database to update and synchronize player scores instantly.

Leaderboard Functionality-

- Global and friend-based leaderboards were implemented, dynamically displaying the highest scores among players.

Cloud Storage and Data Synchronization-

- Player progress, including level completions and experience points, was stored in the cloud, ensuring persistence across sessions and devices.

2.3.3 User Interface and Experience

Main Menu and Navigation-

- The main menu was designed with a themed background and overlay effects, incorporating buttons for gameplay, profile management, and exiting the game.

In-Game Controls-

- A virtual joystick and jump button were implemented to facilitate intuitive control over the character, with real-time updates managed through the game's update loop.

HUD and Score Display-

- A dedicated score display component was integrated into the game's camera viewport, ensuring that players could view their scores dynamically as they progressed.

Profile and Leaderboard Screens-

- Profile screens were developed to display user information, including names, high scores, and experience points.
- A leaderboard interface provided an overview of top scores, enhancing competitive engagement.

Completion and Level Transition-

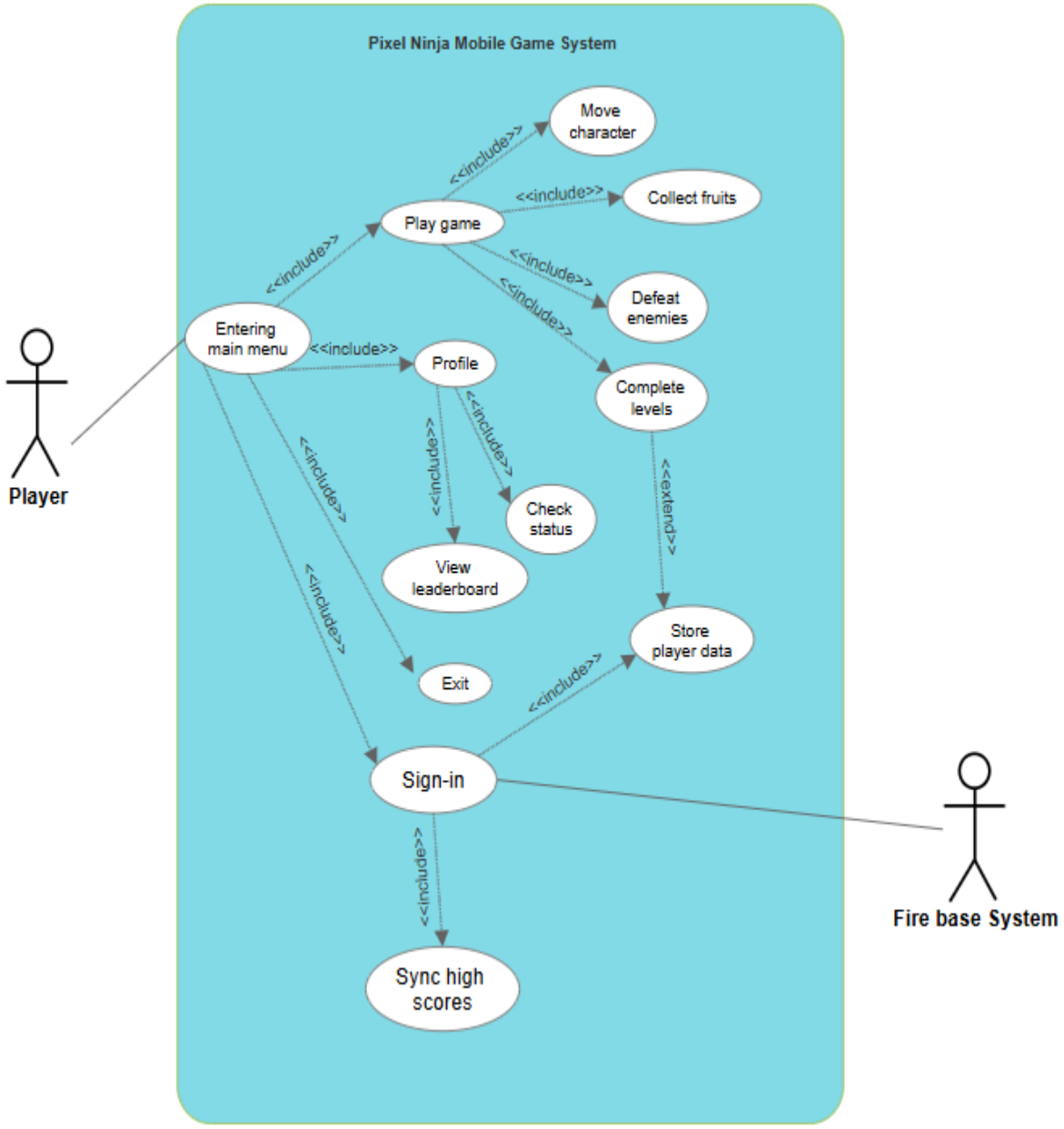
- A game completion screen was provided to display final scores and level information, with options to return to the main menu or proceed to the next level.

Theming and Visual Consistency-

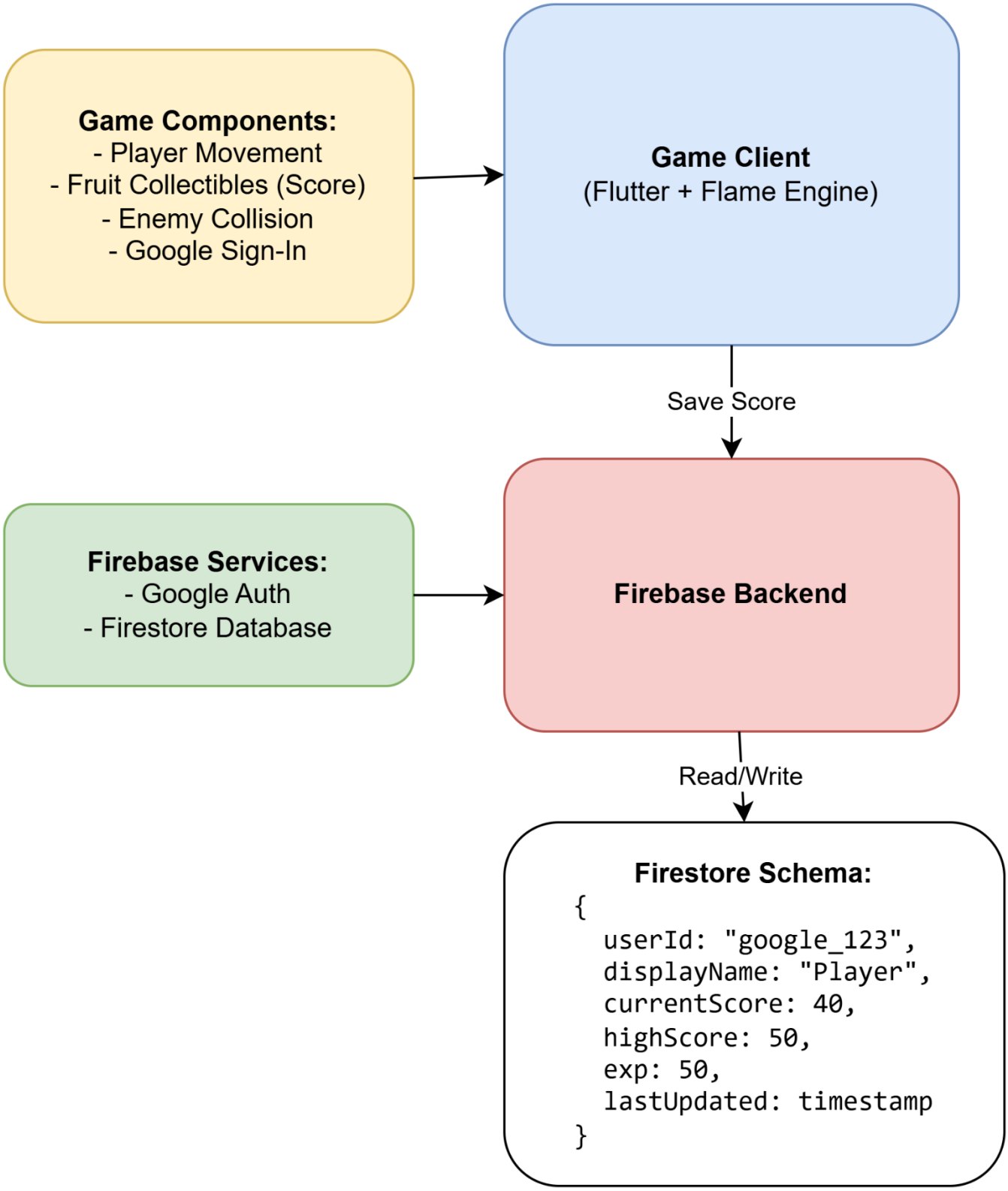
- The user interface employed a dark, pixel-art-inspired theme consistent with the overall game aesthetic, using custom fonts and shadow effects to enhance readability and visual appeal.

Chapter 03

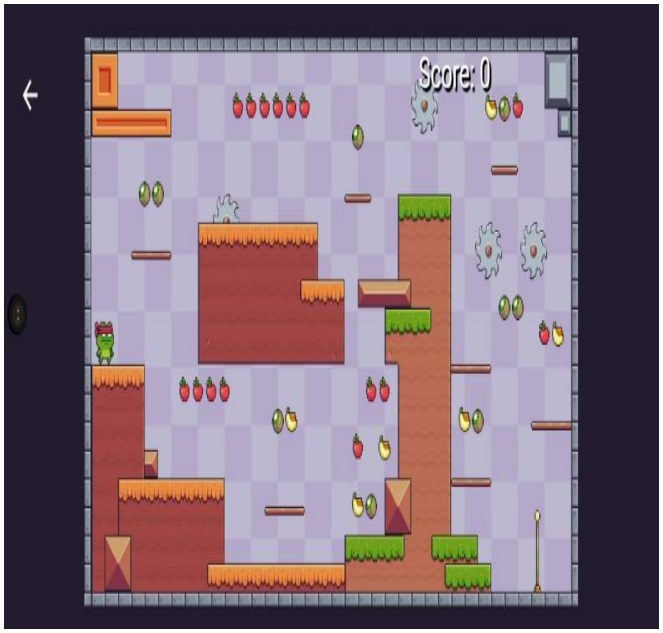
3.1 Use Case Diagram



3.2 High-Level System Architecture



3.4 User interface of the developed System



Chapter 04

4.1 Development Methodology

Agile Scrum was implemented in bi-weekly sprints. Development was divided into three main stages:

Core Gameplay Implementation (Sprints 1-3)

- Player movement was developed in pixel_game.dart, implementing joystick control and physics calculations. Enemy behaviors were implemented in chicken.dart using a state machine (ChickenState) combined with pathfinding computations. Collision detection was iteratively developed in utils.dart.

Firebase Integration (Sprints 4-5)

- Authentication was done via auth_service.dart using Google Sign-In. Score persistence was accounted for in fruit.dart using Firestore batch writes, while experience was tabulated in complete.dart using FieldValue.increment.

Polishing and Optimization (Sprints 6-8)

- The checkpoint system was enhanced within checkpoint.dart with three refinement sprints. Optimizations for performance were done in preloading assets and implementation of delta time. Sound management was done through FlameAudio.

- **Testing Methodology**

  - Unit tests targeted the score systems (fruit.dart).
  - Integration tests considered the player-enemy interactions.
  - Performance tests analyzed memory, CPU usage.

4.2 Technologies and Tools Used

Game Engine Framework

- The **Flame engine (v1.6.0)** was implemented as the core framework , handling physics, collision, and animation. Key implementations included the game loop (pixel_game.dart) and the entity-component architecture (player.dart).

Cross-Platform Development

- **Flutter (v2.10.0)** enabled consistent UI across platforms, powering menus (mainmenu.dart) and in-game overlays (score_display.dart).

Backend Services

- **Firebase services** were integrated to handle cloud-based functionalities. Authentication was managed through auth_service.dart, while **Firestore** was employed for real-time data persistence, particularly for score tracking in fruit.dart and player progression in complete.dart.

Physics and Collision Systems

- A custom physics engine was developed using Flame components. **Collision detection** being handled through utils.dart. The system incorporated **delta-time calculations** in player.dart to ensure frame-rate independent movement and precise hitbox interactions (custom_hitbox.dart).

Audio Management and Assets

- **Flame Audio** managed sound effects throughout the game, While **Asprite** created all pixel-art assets.

Asset Creation and Management

- **Aseprite** served as the primary tool for creating and editing pixel-art assets. All character sprites, enemy animations, and environmental elements were designed using this software before being integrated into the game's asset pipeline.

Level Design Tools

- The **Tiled Map Editor** was employed for designing game levels, with level.dart parsing the exported TMX files to dynamically spawn game objects.

Development Environment

- **Android Studio** and **VS Code** were used for development while Version control was managed through **GitHub**.

4.3 Future Implementation

Core Gameplay Improvements

- The combat system will be enhanced through the implementation of temporary **power-ups** and **new enemy types** with smarter AI. Everything will be integrated smoothly into our existing physics and movement systems, so it feels natural and responsive.

Real-Time Multiplayer & Online Competition

- The **online multiplayer** functionality will be enhanced through the implementation of **Firebase-powered leaderboards** and persistent progression so real-time matchups will be optimized to ensure competitive integrity while maintaining low-latency performance.

Technical Upgrades for a Smoother Gaming Experience

- **Offline play** functionality will be implemented with local progress to be accessed even without Wi-Fi. The game will also **scale the difficulty according to the player's skill level** in terms of how well they play. Cross-platform optimizations will be deployed to ensure consistent performance across iOS and WebGL environments.

Connect and Share More

- New exciting **social features** will be rolled out for Gaming being much better when played together: distinct **achievements** will be showcased, **replays** will be saved (even down to the single frame!), and **custom challenges** will be created for the community. It all build on the back of the current login system, so diving right into the social stuff should be smooth.

Performance Matters

- Every new feature will be put through rigorous tests: **tight memory usage** will be maintained, the snappiness of **online play**, and controls will be made ultra-responsive. These enhancements will preserved **Pixel Ninja** true to its roots while boundaries are pushed, with mobile performance and backward compatibility in mind.

Chapter 05

5.1 Individual Contribution(in paragraph form for everyone)

Karunanayaka Jayawardhana (10952756)

Jayawardhana was responsible for implementing the core functionality of the game, including managing the game image cache, animations, and player states. Additionally, he integrated game sound effects to improve the player experience, ensuring smooth transitions and responsive actions within the game.

Hewawasam Bathika (10952760)

Bathika handled the scoring system and integrated Google login authentication using Firestore. The score system updates dynamically as players collect fruits, and the authentication ensures that user progress, including scores and levels, is securely stored and retrieved from Firestore.

Liyana Jayasinghe (10952746)

Jayasinghe designed and implemented the Main Menu and Profile Page. The Main Menu provides users with options to start the game or navigate different sections, while the Profile Page displays the player's name, current level, total score, and highest score, ensuring an engaging user experience.

Don Hettiarachchi (10952715)

Hettiarachchi developed the Complete Page, level load mechanism, and UI designs. The Complete Page appears when a level is successfully finished, while the level-loading functionality ensures a smooth transition between different stages of the game. The UI designs enhance the visual appeal of the game.

Asini Aththanayake (10955156)

Asini implemented the traps mechanics and designed their appearance. This included programming the behavior of obstacles like saws and spikes, ensuring they function correctly within the game environment. The trap mechanics add difficulty and make the gameplay more challenging.

Withana Gayuka (10952778)

Gayuka was responsible for collectible items and background design. He implemented the logic for collecting fruits to increase the player's score and designed the background elements to enhance the visual aesthetics of the game. His contributions helped create an interesting game environment.

Jothikalananthan Janusha (10952971)

Janusha developed the jump functionality, collision detection, and various design elements. The jump button is crucial for navigating obstacles, while the collision mechanics ensure accurate interactions between the player, traps, and collectibles.

Thennakoon Thennakoon (10953085)

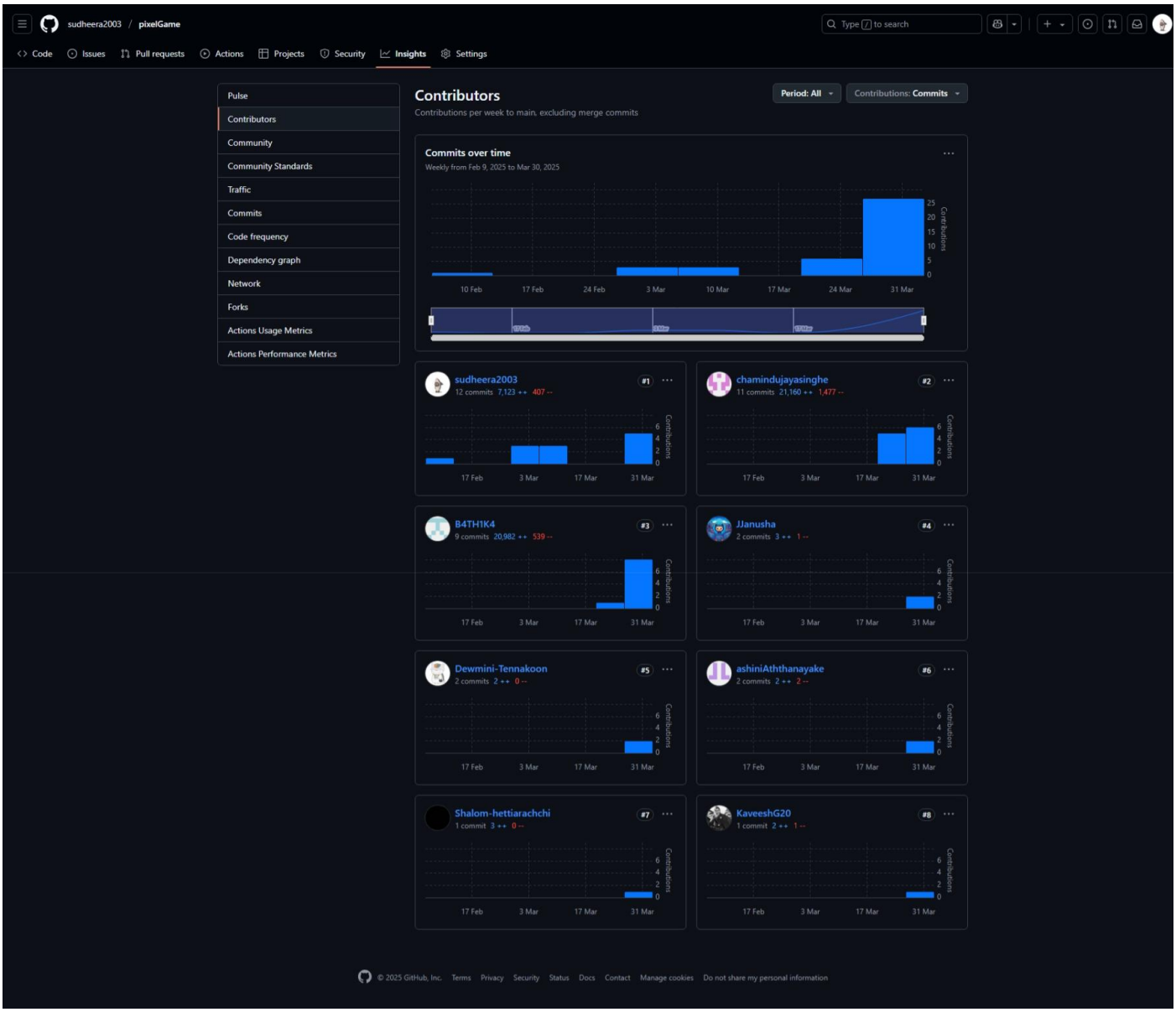
Thennakoon implemented the finishpoint system and additional UI designs. The finishpoint feature allows players to pass to next level from a currently playing level.

5.2 Github Commit History along with Github repository link

GitHub Repository Link - https://github.com/sudheera2003/pixelGame.git

Commits Log –

Member	Plymouth index no	Name	GitHub Username
1	10952760	Hewawasam Bathika	B4TH1K4
2	10952715	Don Hettiarachchi	Shalom-hettiarachchi
3	10952756	Karunanayaka Jayawardhana	sudheera2003
4	10952746	Liyana Jayasinghe	chamindujayasinghe
5	10955156	Asini Aththanayake	
6	10952778	Withana Gayuka	KaveeshG20
7	10952971	Jothikalananthan Janusha	JJanusha
8	10953085	Thennakoon Thennakoon	Dewmini-Tennakoon


sudheera2003 / pixelGame
Type / to search
Code
Issues
Pull requests
Actions
Projects
Security
Insights
Settings
Pulse
Contributors
Community
Community Standards
Traffic
Commits
Code frequency
Dependency graph
Network
Forks
Actions Usage Metrics
Actions Performance Metrics
Contributors
Contributions per week to main, excluding merge commits
Period: All
Contributions: Commits
Commits over time
Weekly from Feb 9, 2025 to Mar 30, 2025
10 Feb
17 Feb
24 Feb
3 Mar
10 Mar
17 Mar
24 Mar
31 Mar
Contributions
sudheera2003
12 commits 7,123 ++ 407 --
#1
chamindujayasinghe
11 commits 21,160 ++ 1,477 --
#2
B4TH1K4
9 commits 20,982 ++ 539 --
#3
JJanusha
2 commits 3 ++ 1 --
#4
Dewmini-Tennakoon
2 commits 2 ++ 0 --
#5
ashiniAththanayake
2 commits 2 ++ 2 --
#6
Shalom-hettiarachchi
1 commit 3 ++ 0 --
#7
KaveeshG20
1 commit 2 ++ 1 --
#8
17 Feb
3 Mar
17 Mar
31 Mar
© 2025 GitHub, Inc.
Terms
Privacy
Security
Status
Docs
Contact
Manage cookies
Do not share my personal information

5.3 Reference List

[1]**Flutter,** "Flutter Documentation," Available: https://docs.flutter.dev/. [Accessed: Apr. 4, 2025].

[2] **Flame Engine,** "Flame Game Engine Documentation," Available: https://docs.flame-engine.org/latest/. [Accessed: Apr. 4, 2025].

[3] **Itch.io,** "Game Assets Marketplace," Available: https://itch.io/. [Accessed: Apr. 4, 2025].

[4] **JSFXR,** "Online Sound Effect Generator for Games," Available: https://sfxr.me/. [Accessed: Apr. 4, 2025].